

Trabajo Práctico - Programación P1

- Super Elizabeth Sis

Participantes:

Zoe Jaimez - 45307959

zoeaj4@gmail.com

Leandro Villalba - 47233518

lean9villalba@gmail.com

Introducción

Super Elizabeth Sis es un juego de plataformas 2D desarrollado en Java. La protagonista, Elizabeth, debe recorrer un nivel de 5000 píxeles de ancho, eliminar enemigos, recolectar pociones y llegar al castillo del extremo derecho para ganar. El proyecto se organiza en siete clases: Juego, Elizabeth, Enemigo, Isla, Proyectoil, Mapa y Poción.

Este informe describe cómo fue pensada y construida cada clase, qué información guarda, qué hace cada método y cómo se relaciona con el resto del sistema. Al final se documentan los principales problemas que surgieron durante el desarrollo y cómo se resolvieron.

Clase Juego

Qué hace

Se encarga de coordinar el ciclo de actualización (tick), gestionar el estado del juego, controlar cuántos enemigos y pociones hay en pantalla y mostrar la pantalla de fin de partida.

Datos que guarda

Guarda una referencia a cada objeto activo: la princesa, el mapa, el proyectil en vuelo y dos arreglos de tamaño fijo para los enemigos y las pociones. Una posición en null dentro de esos arreglos significa que el lugar está libre para ser reutilizado. También almacena el número de vidas y el estado actual del juego (jugando, perdido o ganado). Además contiene el tick().

Cómo funciona el ciclo tick()

El motor llama a tick() aproximadamente 60 veces por segundo. Al comienzo del tick se revisa si el juego ya terminó: si es así, se muestra la pantalla final y no se hace nada más. Si se está jugando, se ejecutan en orden los siguientes pasos:

- Se dibuja el mapa (fondo, castillo e islas).
- Se leen las teclas y se mueve a la princesa.
- Se calcula el desplazamiento de la cámara.
- Se aplica gravedad y se detectan colisiones con las islas.
- Se procesa el disparo.
- Se generan nuevos enemigos si corresponde.
- Se mueven, colisionan y dibujan los enemigos.
- Se mueven, colisionan y dibujan las pociones.
- Se dibuja la princesa y el proyectil encima de todo.
- Se evalúan las condiciones de victoria o derrota.

Sistema de máximo de enemigos con slots null

Para limitar la cantidad de enemigos en pantalla sin estructuras de datos complejas, se usa un arreglo de tamaño fijo (`MAX_ENEMIGOS = 8`). Cada posición puede estar ocupada por un enemigo o estar vacía (`null`). Cuando un enemigo muere o sale de la ventana, su posición se pone en `null`, liberando el lugar para el próximo.

La aparición de nuevos enemigos sigue dos reglas: si hay menos de tres enemigos vivos se genera uno de inmediato, o si pasaron más de 90 ticks desde el último spawn también se genera uno. En ambos casos se busca el primer slot `null` y se coloca ahí el nuevo enemigo. Si el arreglo está lleno, no aparece ninguno más.

Pantalla final y reinicio

Al ganar o perder, se muestra la imagen correspondiente y un botón "Jugar de nuevo". Al hacer clic dentro del área del botón se recrean todos los objetos desde cero y el estado vuelve a JUGANDO, sin cerrar la ventana.

Clase Elizabeth

Qué hace

Elizabeth representa a la protagonista. Guarda su posición en el mundo (no en pantalla), su velocidad vertical, si está en el aire, hacia dónde mira y sus cuatro imágenes posibles.

Movimiento horizontal

Moverse es simple: `moverDerecha()` suma 4 píxeles a `x`, y `moverIzquierda()` los resta, con un tope mínimo en `x = 0` para que no salga por el borde izquierdo del mundo.

Gravedad y salto

La gravedad es un valor constante (1.0) que se suma a `velocidadY` en cada tick. Esa `velocidadY` se suma a `y`. Como se acumula tick a tick, la caída se acelera de forma natural, igual que en la física real.

Al saltar, `velocidadY` se fija en -18 (negativo porque `y` crece hacia abajo en pantalla). La gravedad va frenando ese impulso hasta que `velocidadY` vuelve a ser positiva y la princesa empieza a caer. Solo se puede saltar si `enElAire` es `false`, evitando el doble salto.

Aterrizaje y techo

Cuando desde Juego se detecta que la princesa pisó una isla, se llama a `aterrizar(ySuperficie)`, que coloca a la princesa con los pies en el tope de la isla y pone `velocidadY` en 0. Si la cabeza choca contra el fondo de una isla, `pegarTecho(yFondo)` la empuja hacia abajo y corta el impulso vertical.

Imágenes según estado

Al dibujarse se elige entre cuatro imágenes: si está en el aire usa la imagen de salto, y según hacia dónde mira usa la imagen de derecha o izquierda. La posición en pantalla se calcula restando el desplazamiento del mapa antes de dibujar.

Clase Enemigo

Qué hace

Enemigo modela a los personajes que atraviesan la pantalla de lado a lado. Si tocan a la princesa le quitan una vida. Si son eliminados por el proyectil, tienen un 15% de probabilidad de soltar una poción.

```
public Poción soltarPoción(double desplazamientoMundoX) {  
    if (Math.random() < 0.15) { // — acá  
        return new Poción(x + desplazamientoMundoX, y); // crea  
        la poción en el mundo  
    }  
    return null;  
}
```

Creación y dirección

El constructor recibe un booleano `desdeDerecha` y una altura aleatoria. Si viene de la derecha, `x` empieza en 810 (fuera de pantalla) y la velocidad es -5. Si viene de la izquierda, `x` empieza en -10 y la velocidad es +5. Así el enemigo siempre entra desde afuera y cruza la pantalla.

Coordenadas de pantalla!!!

A diferencia de la princesa y las pociones, el enemigo vive en coordenadas de **pantalla** (entre 0 y 800). Esto simplifica su lógica porque no necesita saber nada del desplazamiento del mapa. La consecuencia es que al compararlo con la princesa hay que convertir la posición de la princesa a pantalla primero, lo que se hace en Juego.

Colisiones

La detección de colisión usa cajas rectangulares centradas en cada objeto. Para que dos objetos colisionen, la distancia entre sus centros en X debe ser menor que la suma de sus medios anchos, y lo mismo en Y. Para el proyectil se agrega un margen extra de 15 píxeles para que sea más fácil acertar.

Soltar poción

Cuando el proyectil mata al enemigo, `soltarPoción(desplazamientoMundoX)` tiene un 15% de probabilidad de crear una poción. El `desplazamientoMundoX` (el desplazamiento de la cámara)

se suma a la posición de pantalla del enemigo para convertirla a coordenada mundo, que es donde vive la poción.

Clase Isla

Qué hace

Isla representa cada plataforma del nivel. Guarda su posición y tamaño en coordenadas de mundo, y expone los métodos `tope()` y `fondo()` que se usan para detectar colisiones verticales con la princesa y las pociones.

`tope()` y `fondo()`

Como el objeto está centrado en y, el borde superior (`tope`) es `y - alto/2` y el borde inferior (`fondo`) es `y + alto/2`. Estos valores son los que Juego compara con los pies y la cabeza de la princesa para decidir si aterriza o choca contra el techo.

Dibujo con scroll

Al dibujarse, se convierte la posición de mundo a pantalla restando el desplazamiento. Además se verifica si la isla está dentro de la ventana antes de dibujarla: si está completamente fuera (por cualquiera de los dos lados) se salta el dibujo para no hacer trabajo innecesario. Si la imagen no cargó, se dibuja un rectángulo verde como respaldo visual.

Clase Proyectoil

Qué hace

Proyectoil representa el disparo que lanza la princesa. Solo puede existir uno a la vez: mientras haya un proyectoil en vuelo no se puede disparar otro. Al salir de pantalla o golpear un enemigo, se destruye (se pone en null en Juego).

Dirección y movimiento

En el constructor se calcula el ángulo hacia el cursor con `Math.atan2(destinoY - y, destinoX - x)`. Cada tick, `moverse()` avanza el proyectoil multiplicando el coseno del ángulo por la velocidad (componente X) y el seno por la velocidad (componente Y), produciendo movimiento recto hacia donde apuntaba el mouse.

```
// destinoX, destinoY = posición del cursor en el momento del
disparo. atan2 calcula el ángulo entre el origen del disparo y el
destino
public Proyectil(double x, double y, double destinoX, double
destinoY) {
    this.x = x;
    this.y = y;
    this.angulo = Math.atan2(destinoY - y, destinoX - x);
}
```

Coordenadas de pantalla

El proyectil vive en coordenadas de pantalla. Al crearlo, Juego convierte la posición de la princesa de mundo a pantalla antes de pasarla al constructor, para que el punto de origen del disparo coincida con lo que se ve en la ventana.

Clase Mapa

Mapa es el contenedor del nivel. Genera todas las islas de forma aleatoria al inicio, guarda el fondo y el castillo, y calcula el desplazamiento de la cámara en cada tick para que la princesa quede centrada en pantalla.

Generación de islas

Las islas se crean en tres grupos dentro de generarIslas():

- Piso (15 islas): se ubican a $y = 565$, con un ancho fijo de 200 píxeles y huecos de 150 píxeles entre ellas. No hay aleatoriedad: siempre quedan en el mismo lugar.
- Nivel 1 (10 islas): distribuidas uniformemente a lo largo del mundo, con variaciones aleatorias de ± 50 píxeles en X, ± 20 en Y y ancho entre 120 y 200 píxeles. Se ubican a unos 400 píxeles de altura.
- Nivel 2 (10 islas): igual que el nivel 1 pero a unos 250 píxeles de altura, creando plataformas más altas para explorar.

La aleatoriedad en posición y ancho hace que cada partida tenga un nivel ligeramente distinto.

Cálculo del desplazamiento

El método `actualizarDesplazamiento(xPrincesa)` calcula cuántos píxeles hay que correr la cámara: `desplazamiento = xPrincesa - 400`. El resultado se limita entre 0 y 4200 (= 5000 - 800) para que la cámara nunca muestre el vacío más allá de los bordes del mundo. La cámara es unidireccional: el desplazamiento solo puede aumentar, nunca disminuir. Esto significa que cuando la princesa avanza a la derecha la cámara la sigue, pero si retrocede hacia la izquierda la cámara se queda quieta, igual que en los juegos de Mario Bros.

```
public void actualizarDesplazamiento(double princesaXMundo)
{
    double mitadPantalla = ANCHO_PANTALLA / 2.0;
    double nuevoDesplazamiento = princesaXMundo -
    mitadPantalla;
    double maxDesplazamiento = ANCHO_MUNDO -
    ANCHO_PANTALLA;
    if (nuevoDesplazamiento < 0)        nuevoDesplazamiento =
    0;
    if (nuevoDesplazamiento > maxDesplazamiento)
    nuevoDesplazamiento = maxDesplazamiento;
    // La cámara solo avanza, nunca retrocede
    if (nuevoDesplazamiento > this.desplazamientoMapaX) {
        this.desplazamientoMapaX = nuevoDesplazamiento;
    }
}
```

Dibujo

El fondo siempre se dibuja centrado en la ventana y no se mueve con el scroll. El castillo y las islas sí se desplazan: cada uno recibe el valor de desplazamiento y resta ese valor a su coordenada de mundo para obtener la posición en pantalla.

Clase Poción

Qué hace

Poción es el objeto de recompensa que puede soltar un enemigo al morir. Al recogerla, la princesa gana una vida. Existe en coordenadas de mundo y cae con su propia gravedad hasta posarse en una isla o caer al vacío.

Física propia

La poción tiene una gravedad más suave que la princesa (0.4 en vez de 1.0) y empieza con velocidad vertical negativa (-3) para hacer un pequeño rebote hacia arriba al aparecer. Luego cae hasta tocar una isla o salir del mapa.

Colisión con islas

El método `moverse(islas[])` recibe el arreglo de islas del mapa. En cada tick, después de aplicar la gravedad, revisa si la poción aterrizó sobre alguna isla.

Dibujo y colisión con la princesa

Para dibujar, se resta el desplazamiento de la cámara a su coordenada de mundo, igual que con las islas. La colisión con la princesa se detecta directamente en coordenadas de mundo porque ambas viven en el mismo sistema, sin necesidad de conversiones.

Clase ProyectoilArea

Que hace

ProyectoilArea representa el disparo secundario de la princesa. A diferencia del proyectil normal, este no elimina solamente al primer enemigo, al colisionar, genera una “explosión” en área que destruye a todos los enemigos dentro de un radio determinado. ProyectoilArea mantiene la misma lógica del disparo normal: solo puede existir un disparo secundario a la vez, y además no se puede disparar si ya existe otro proyectil activo (o sea no puedo disparar proyectil normal y en área a la vez) esto se controla verificando `disparo == null && disparoArea == null`

Dirección y movimiento

El proyectil secundario reutiliza la misma lógica matemática del disparo normal

Explosión en área

Cuando el proyectil toca un enemigo, desde Juego se llama al método `explotar(disparoArea)`. Este método recorre el arreglo completo de enemigos y calcula la distancia entre cada enemigo y el centro de la explosión. Si la distancia es menor o igual al radio de la explosión, el enemigo

muere.

La fórmula `Math.sqrt(dx*dx + dy*dy)` es la distancia entre dos puntos en el plano cartesiano. Se usa porque permite detectar enemigos en cualquier dirección alrededor del impacto, formando un área circular.

```
// distancia entre enemigo y centro de explosión
double dx = enemigos[i].getX() - p.getX();
double dy = enemigos[i].getY() - p.getY();
double distancia = Math.sqrt(dx*dx + dy*dy);
// si está dentro del radio muere
if (distancia <= p.getRadioExplosion()) {
    enemigos[i] = null;
}
```

Clase JefeFinal

Que hace

La clase JefeFinal representa al enemigo principal del juego, que aparece cuando la protagonista llega al castillo ubicado al final del mapa. Es el último desafío antes de alcanzar la condición de victoria.

Variables de instancia:

`double x`: posición horizontal del jefe.

`double y`: posición vertical del jefe.

`int vidas`: cantidad de puntos de vida del jefe. Inicialmente posee 20 vidas.

`Image imagen`: imagen utilizada para representar gráficamente al jefe en pantalla.

Métodos implementados

`JefeFinal(double x, double y)`

Constructor de la clase. Inicializa la posición del jefe, establece su cantidad de vidas y carga la imagen correspondiente desde un archivo GIF.

```
dibujarse(Entorno entorno)
```

Dibuja la imagen del jefe final en pantalla utilizando las herramientas provistas por la librería Entorno.

```
getVidas()
```

Devuelve la cantidad actual de vidas del jefe.

```
recibirDanio()
```

Reduce en una unidad la cantidad de vidas del jefe cada vez que recibe el impacto de un proyectil.

```
estaMuerto()
```

Verifica si la cantidad de vidas llegó a cero o menos. Devuelve verdadero cuando el jefe ha sido derrotado.

```
colisionaCon(double px, double py)
```

Determina si un proyectil impactó contra el jefe mediante una comprobación simple de distancia horizontal y vertical respecto de su posición.

Integración del Jefe Final en la clase Juego

El jefe final se encuentra integrado dentro de la lógica principal del juego mediante la variable `private JefeFinal jefe;`

Para lograr que cuando Elizabeth llegue al castillo y spawnee el jefe final, se implementó un nuevo estado JEFE_FINAL.

```
private static final int JUGANDO = 0;
private static final int JEFE_FINAL = 1;
private static final int PERDIDO = 2;
private static final int GANADO = 3;
```

Cuando la princesa alcanza la posición del castillo, el estado del juego cambia a JEFE_FINAL

```
// Llegó al castillo, el estado cambia a jefe final,
se eliminan enemigos
if (estado == JUGANDO && princesa.getX() >=
mapa.getXCastillo() - 80) {
```

```

        estado = JEFE_FINAL;
        jefe = new JefeFinal(400, 100);
        // eliminar todos los enemigos
        for (int i = 0; i < enemigos.length; i++){
            enemigos[i] = null;
        }
    }
}

```

Los enemigos comunes son eliminados para evitar interferencias durante el combate final.

Durante este estado, los proyectiles pueden causar daño al jefe. Cada impacto ejecuta el método `jefe.recibirDanio()`;

Finalmente cuando la cantidad de vidas llega a cero, el estado del juego cambia a GANADO, mostrando la pantalla de victoria.

Relaciones entre las clases

Juego es el núcleo que contiene y coordina a todos los demás. Elizabeth, Enemigo, Proyectil y Poción son objetos que saben moverse y dibujarse pero no conocen a los otros. Isla y Mapa forman el escenario: Mapa genera y agrupa las Islas y gestiona el scroll.

La distinción más importante del diseño es el espacio de coordenadas. Elizabeth, Isla, Mapa y Poción viven en coordenadas de mundo (0 a 5000 en X). Enemigo y Proyectil viven en coordenadas de pantalla (0 a 800 en X). Juego se encarga de hacer las conversiones necesarias cuando estos dos grupos necesitan compararse entre sí.

Problemas encontrados y cómo se resolvieron

1. La poción quedaba fija en pantalla y se alejaba sola

Cuando un enemigo moría y soltaba una poción, la poción aparecía en la posición de pantalla del enemigo (entre 0 y 800). Mientras la princesa seguía avanzando y el mapa se desplazaba, la poción no se movía: visualmente se iba corriendo hacia la izquierda hasta desaparecer, sin que la princesa pudiera alcanzarla.

La solución fue que la poción guardara su posición en coordenadas de mundo, no de pantalla. En la clase Enemigo, al soltar la poción, se suma el desplazamiento actual de la cámara a la posición del enemigo:

```

return new Poción(x + desplazamientoMundoX, y); // crea la
poción en el mundo

```

Así la poción queda anclada a un punto fijo del mundo y se mueve junto con el scroll de la cámara. Tanto el movimiento como la colisión con la princesa se calculan en coordenadas de

mundo, y solo al dibujarla se convierte a pantalla restando el desplazamiento. Esto se resolvió en las clases Enemigo y Poción.

2. Como hacer que el mapa se mueva con la princesa (scroll de cámara)

El nivel mide 5000 píxeles de ancho pero la ventana solo muestra 800. El problema era cómo lograr que, a medida que la princesa avanza, el escenario la siga sin que ella salga de la ventana.

La solución está en la clase Mapa, en el método actualizarDesplazamiento(). Cada tick se calcula un valor de desplazamiento que representa cuántos píxeles hay que correr la cámara para mantener a la princesa centrada:

```
public void actualizarDesplazamiento(double princesaXMundo)
{
    double mitadPantalla = ANCHO_PANTALLA / 2.0;
    double nuevoDesplazamiento = princesaXMundo -
    mitadPantalla;
    double maxDesplazamiento = ANCHO_MUNDO -
    ANCHO_PANTALLA;
    if (nuevoDesplazamiento < 0)        nuevoDesplazamiento =
    0;
    if (nuevoDesplazamiento > maxDesplazamiento)
    nuevoDesplazamiento = maxDesplazamiento;
    // La cámara solo avanza, nunca retrocede
    if (nuevoDesplazamiento > this.desplazamientoMapaX) {
        this.desplazamientoMapaX = nuevoDesplazamiento;
    }
}
```

Ese valor se limita entre 0 y 4200 para que la cámara nunca muestre el vacío más allá de los bordes del mundo. Además, el desplazamiento **solo puede aumentar**: si la princesa retrocede hacia la izquierda, la cámara se queda quieta. Luego, para dibujar cualquier objeto del mundo en pantalla, se le resta ese desplazamiento a su posición real. Islas, pociones, castillo y la propia princesa usan esta conversión. Los enemigos y el proyectil son la excepción porque viven directamente en coordenadas de pantalla.

4. Las colisiones y verificación de la ubicación de la princesa

Había dos problemas: detectar si la princesa pisó una isla o chocó contra su fondo, y detectar si colisionó con un enemigo o una poción.

Para las islas, la detección tiene dos pasos: primero se verifica si la princesa está alineada horizontalmente con la isla, y solo si lo está se revisa la colisión vertical. Para aterrizar, se comprueba que la princesa esté cayendo y que sus pies hayan cruzado el tope de la isla. Para el techo, que esté subiendo y su cabeza haya tocado el fondo:

```
// Aterrizaje
princesa.getY() + mitadAlto >= isla.tope()
    && princesa.getY() - mitadAlto < isla.tope()

// Techo
princesa.getY() - mitadAlto <= isla.fondo()
    && princesa.getY() + mitadAlto > isla.fondo()
```

Para enemigos y pociones, la colisión compara la distancia entre centros con la suma de sus medios anchos y altos. Un detalle importante: los enemigos viven en coordenadas de pantalla, así que antes de comparar se convierte la posición de la princesa a pantalla restando el desplazamiento. Las pociones en cambio viven en el mundo igual que la princesa, por lo que se comparan directamente. Esto se resuelve en las clases Enemigo, Poción y Juego.

5. Que la princesa llegue al castillo

El castillo está al final del mundo, en la posición 4900 en coordenadas de mundo. El problema era detectar cuándo la princesa llega hasta ahí, considerando que su posición también es en coordenadas de mundo.

La solución es directa: al final de cada tick se compara la posición de la princesa en el mundo con la posición del castillo en el mundo. Como ambos están en el mismo sistema de coordenadas, no hace falta ninguna conversión:

```
if (princesa.getX() >= mapa.getXCastillo() - 80) {
    estado = GANADO;
}
```

El -80 da un margen para que la victoria se detecte cuando la princesa toca visualmente el castillo y no cuando llega exactamente a su centro. Cuando la condición se cumple, el estado pasa a GANADO y el siguiente tick muestra la pantalla final.

6. Cambiar de estado a JEFE_FINAL

Uno de los problemas fue que al llegar al castillo no se cambiaba correctamente el combate contra el jefe, la raíz de este inconveniente surge de que habían dos bloques independientes que verificaban la condición `estado == jefe_final` dentro del método principal `tick()`. Debido a que el primero finalizaba con una instrucción `return`, el segundo nunca llegaba a ejecutarse se solucionó organizando bien el código y cambiando el estado correctamente cuando la princesa toque el castillo.

7. Eliminar enemigos comunes durante el estado JEFE_FINAL

Al llegar al castillo y spawnear el jefe final, los enemigos comunes seguían apareciendo pero al no estar dibujados eran invisibles, lo que hacía que al disparar al jefe, los proyectiles colisionen

con los enemigos comunes y además le hagan daño a la princesa. La solución fue agregar:
`if (estado == jugando)` en el bloque de código de respawn de enemigos.